



nasm 64 bits (suite)
Pile
Orientation little-endian
Multiplication et division

Sommaire

Accès à la mémoire en écriture

[La pile](#)

[Orientation little-endian](#)

[Entiers signés et non signés](#)

[Instructions `imul` et `idiv`](#)

Accès à la mémoire en écriture



```
section .data
height: dw 0
width:  dw 641
section .text
mov word [height], r12w
```

```
add word [height], 48
add word [height+2], 160
inc word [width]
```

Accès à la mémoire en écriture : ici comme
premier opérande d'une instruction

Accès à la mémoire en écriture



`add [height], bx ; 2 octets car bx est sur 2 octets`

Si on ne met pas de spécificateur de taille, la taille de l'accès mémoire est déterminée par la taille de l'autre opérande

Le code est moins lisible

Si rien ne spécifie la taille, c'est une erreur

`add [height], word 160 ; 2 octets`

~~`add [height], 160 ; erreur, taille non spécifiée`~~

~~`dec [rbx] ; erreur, taille non spécifiée`~~

Au plus un accès à la mémoire par instruction

```
mov rbx, qword [r12] ; OK
```

```
mov byte [rbx], 60 ; OK
```

```
mov byte [rbx], byte [r12] ; erreur, 2 accès mémoire
```

On a le droit à un seul accès mémoire par instruction

L'autre opérande doit être un registre ou une constante

Pour transférer une donnée d'une adresse à une autre, il faut passer par un registre et deux instructions

```
mov r13b, byte [r12]
```

```
mov byte [rbx], r13b
```

Sommaire

Accès à la mémoire en écriture

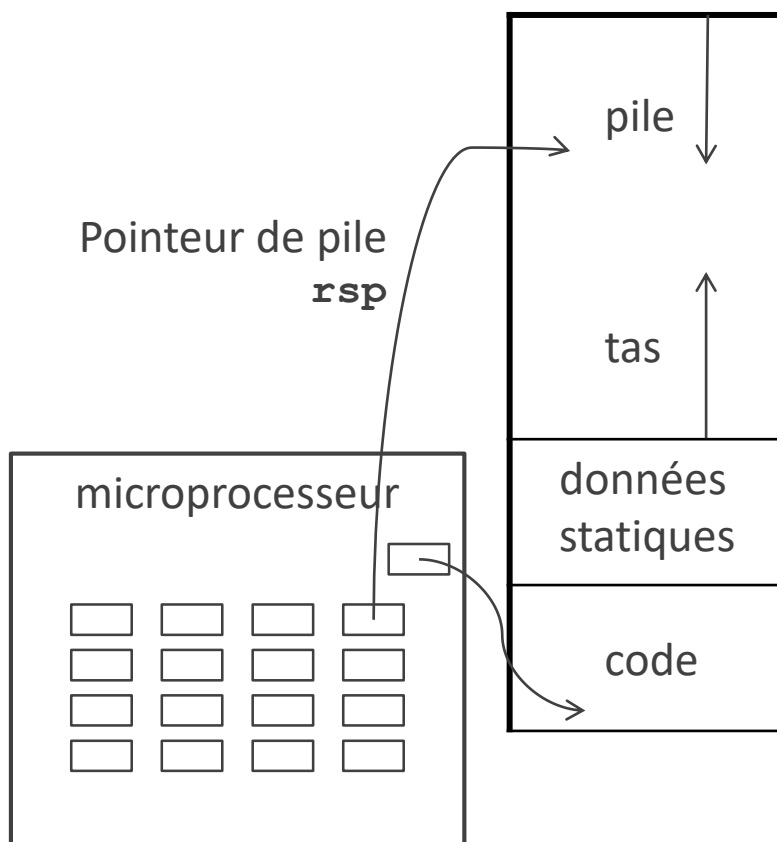
La pile

Orientation little-endian

Entiers signés et non signés

Instructions imul et idiv

Utilisation de la pile



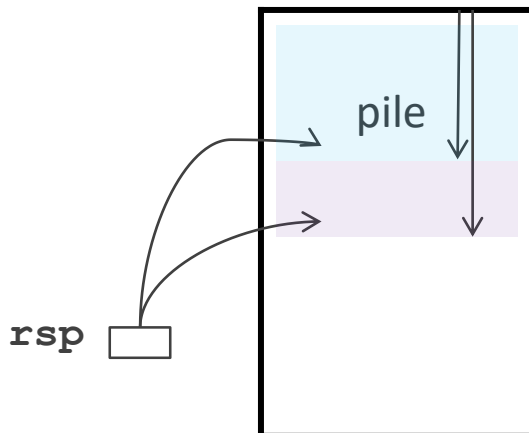
"Fond" de pile dans les adresses hautes

"Sommet" de pile dans les adresses basses

```
mov rbx, 0
push 'g'
push 'u'
push 'y'
pop r12
add rbx, r12
pop r12
add rbx, r12
pop r12
add rbx, r12
call show_registers
```

Instruction **push**

```
push 'u'
push 'y'
pop r12
add rbx, r12
pop r12
add rbx, r12
```



Alloue 8 octets dans la pile

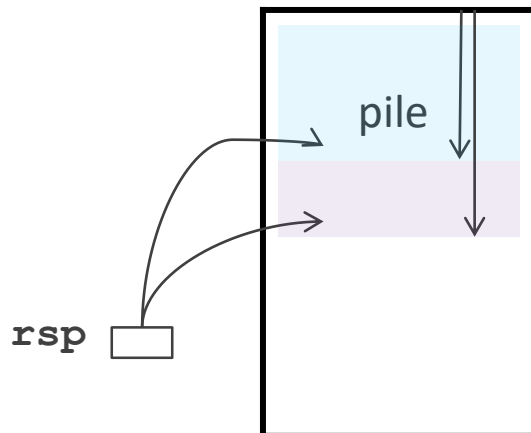
Copie son opérande (8 octets) dans la zone allouée

Déplace **rsp** de 8 octets (décrémententation)

L'opérande peut être une constante, un registre ou un accès mémoire

Instruction `pop`

```
push 'u'
push 'y'
pop r12
add rbx, r12
pop r12
add rbx, r12
```



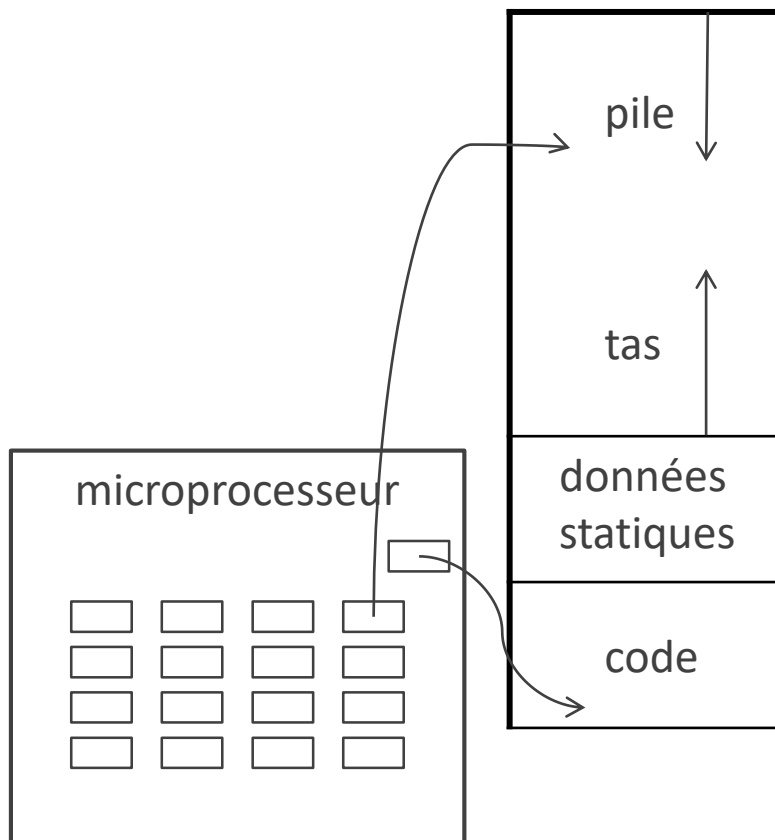
Libère 8 octets dans la pile

Déplace `rsp` de 8 octets (incrémententation)

Copie 8 octets de la pile dans son opérande

L'opérande ne peut pas être une constante

À quoi sert la pile



Fonctions

Pour chaque appel de fonction, un bloc d'activation

Allocation automatique

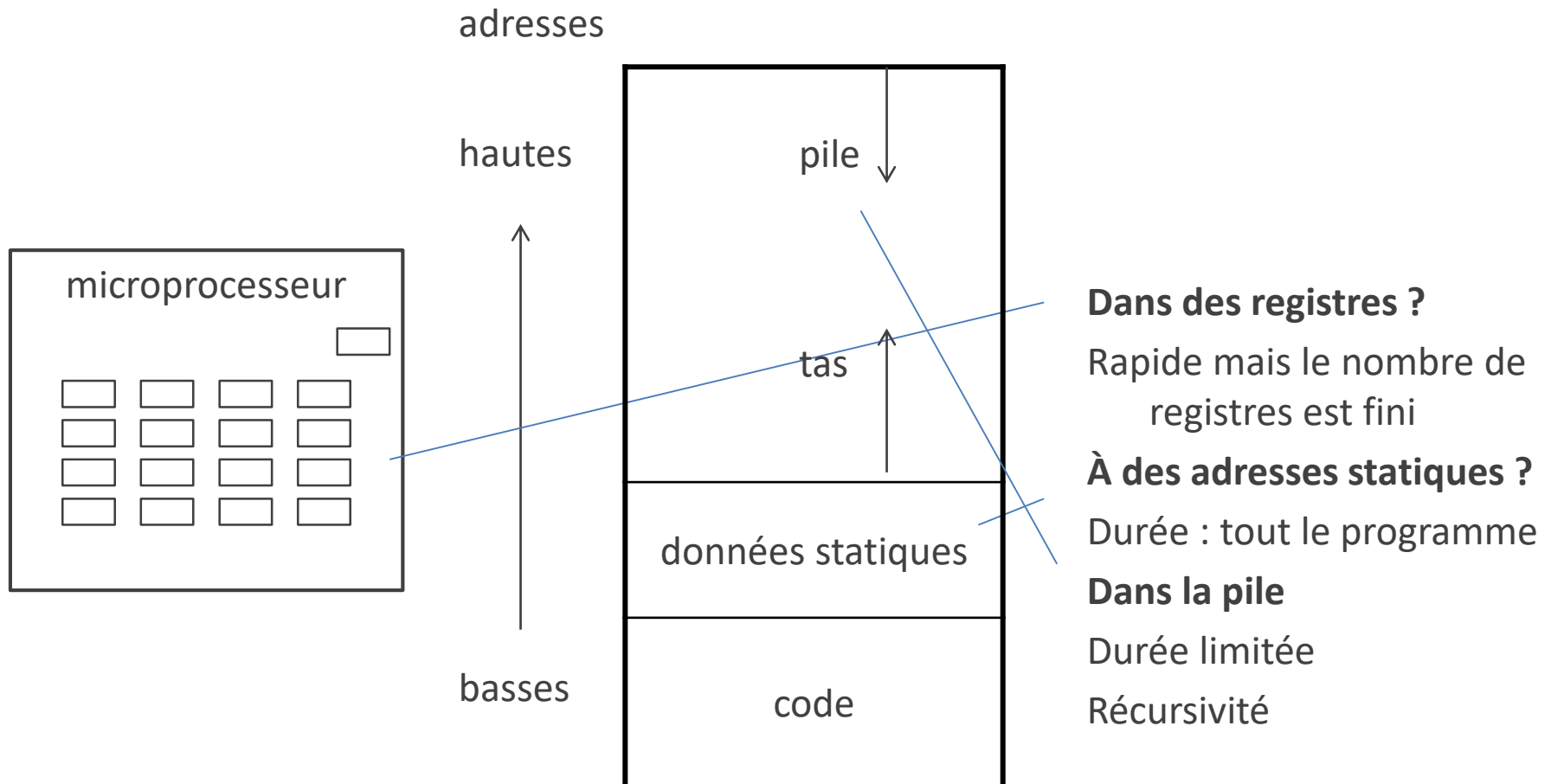
Variables locales (sauf locales statiques)

Résultats intermédiaires des calculs

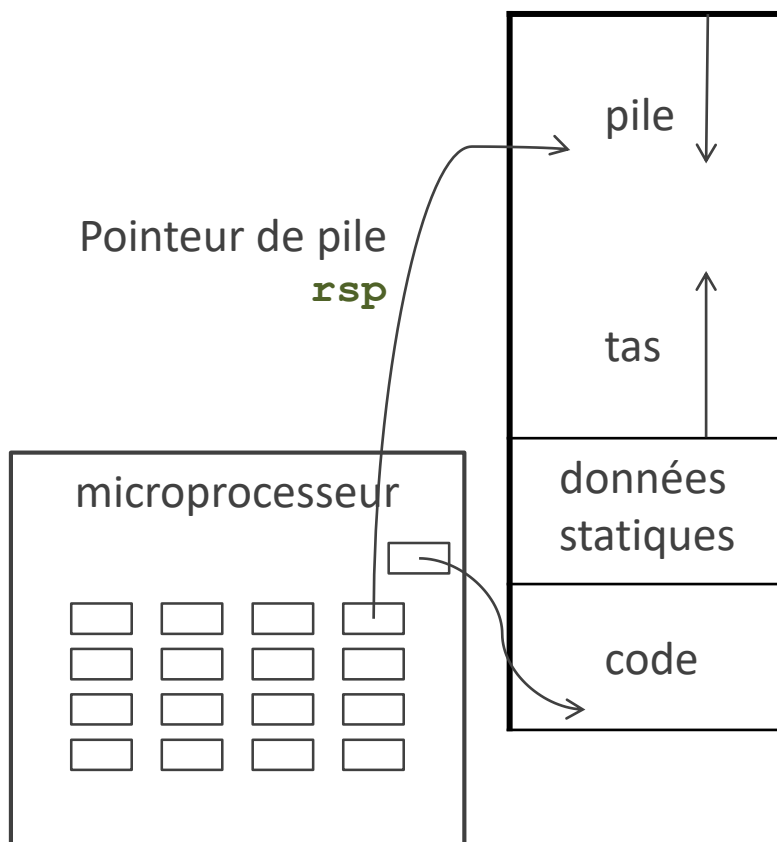
```
int ackerm(int m,int n) {
    if (m==0)
        return n+1;
    if (n==0)
        return ackerm(m-1,1);
    else
        return ackerm(m-1,ackerm(m,n-1));
}
```

nasm • Pile • little-endian • imul • idiv • 10

Sauvegarder les résultats intermédiaires d'un calcul



Le pointeur de pile (*stack pointer*) ou registre **rsp**

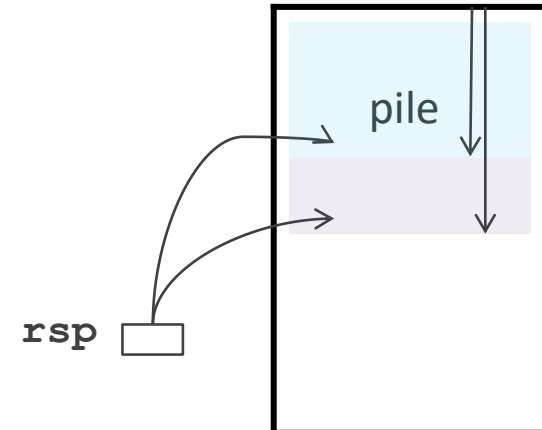


rsp pointe sur l'octet d'adresse la plus basse dans la pile (la dernière donnée empilée)

Accès à la mémoire en pile

```
push 'y'      sub rsp, 8
               mov qword [rsp], 'y'

pop r12       mov r12, qword [rsp]
               add rsp, 8
```



```
push 0x401
push 'star'
mov ebx, dword [rsp]
mov r12d, dword [rsp+4]
mov r13d, dword [rsp+8]
mov r14d, dword [rsp+12]
```

Mêmes règles qu'avec la mémoire statique

Utiliser le registre **rsp**

Permet des accès à des tailles de 1, 2 ou 4 octets,
contrairement à **push** et **pop**

Autre différence : **push** et **pop** n'affectent pas
rflags

Pour prédire ce que contiennent **ebx**, **r12d**,
r13d et **r14d**, il faut savoir où **0x401** et **star**
sont placés dans les 2 zones de 8 octets

Sommaire

Accès à la mémoire en écriture

La pile

Orientation little-endian

Entiers signés et non signés

Instructions `imul` et `idiv`

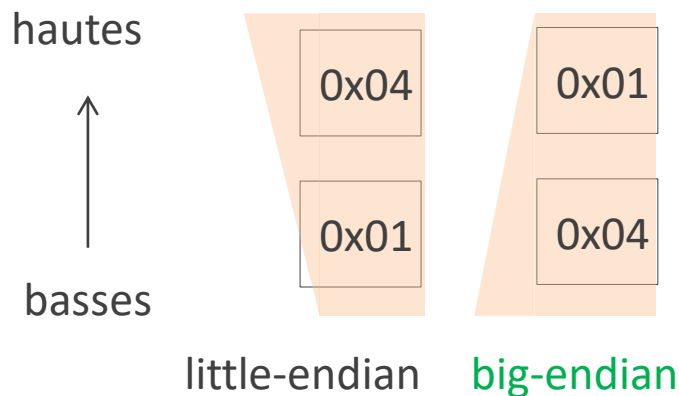
Orientations little-endian et big-endian

décimal : **1025**

hexadécimal : **0x4_01**

binaire : **0b100_0000_0001**

adresses



Une donnée qui nécessite plusieurs octets

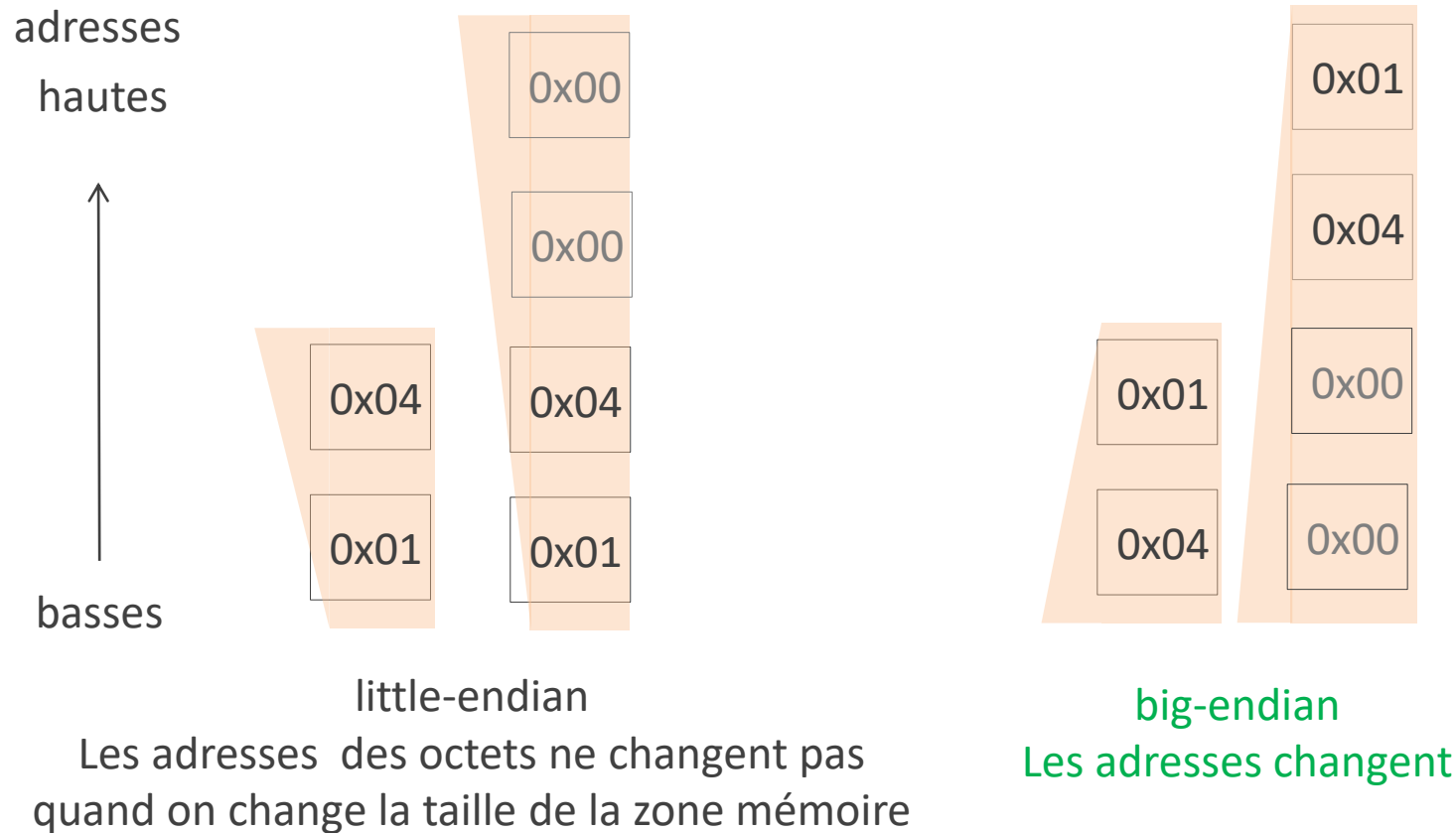
Dans quel ordre sont placés les octets en mémoire ?

On place en premier **soit les chiffres de poids fort (big-endian)** soit ceux de poids faible (little-endian)

décimal : **1025**

hexadécimal : **0x4_01**

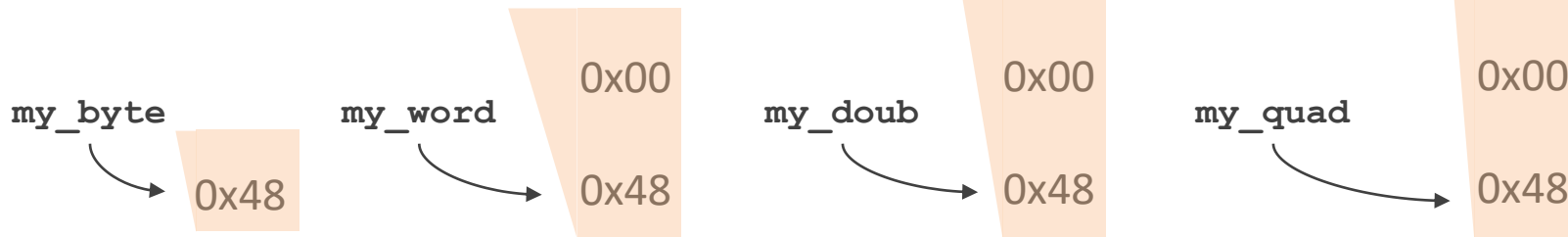
binaire : **0b100_0000_0001**



Orientation little-endian

```
section .data
my_byte: db 72
my_word: dw 72
my_doub: dd 72
my_quad: dq 72
```

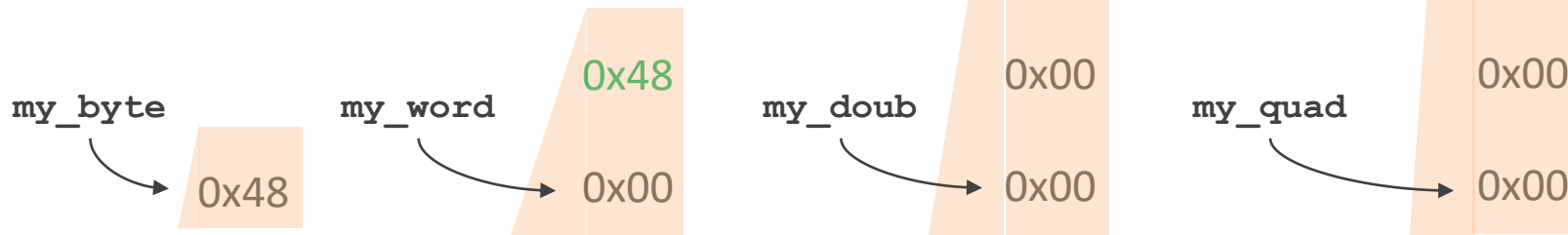
Avec l'orientation little-endian,
l'adresse d'un entier tenant sur un
octet est la même quelle que soit la
taille de mémoire réservée



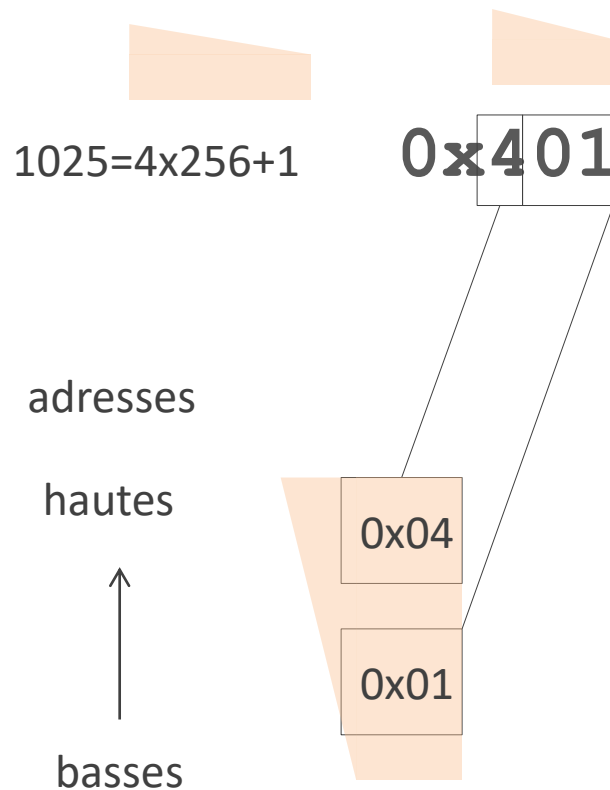
Orientation big-endian

```
section .data
my_byte: db 72
my_word: dw 72
my_doub: dd 72
my_quad: dq 72
```

Avec l'orientation big-endian,
l'adresse d'un entier tenant sur un
octet dépend de la taille de
mémoire réservée



Orientation little-endian



Pour les processeurs x86, la **mémoire** est orientée little-endian

Les octets de poids faible sont aux adresses basses et les octets de poids fort aux adresses hautes
Quand on copie une donnée d'un registre dans la mémoire, l'octet de poids faible va à l'adresse la plus basse

Quand on copie une donnée de la mémoire dans un registre, l'octet à l'adresse la plus basse donne l'octet de poids faible

Notation mathématique et registres


 $1025 = 4 \times 256 + 1$ 
0x401

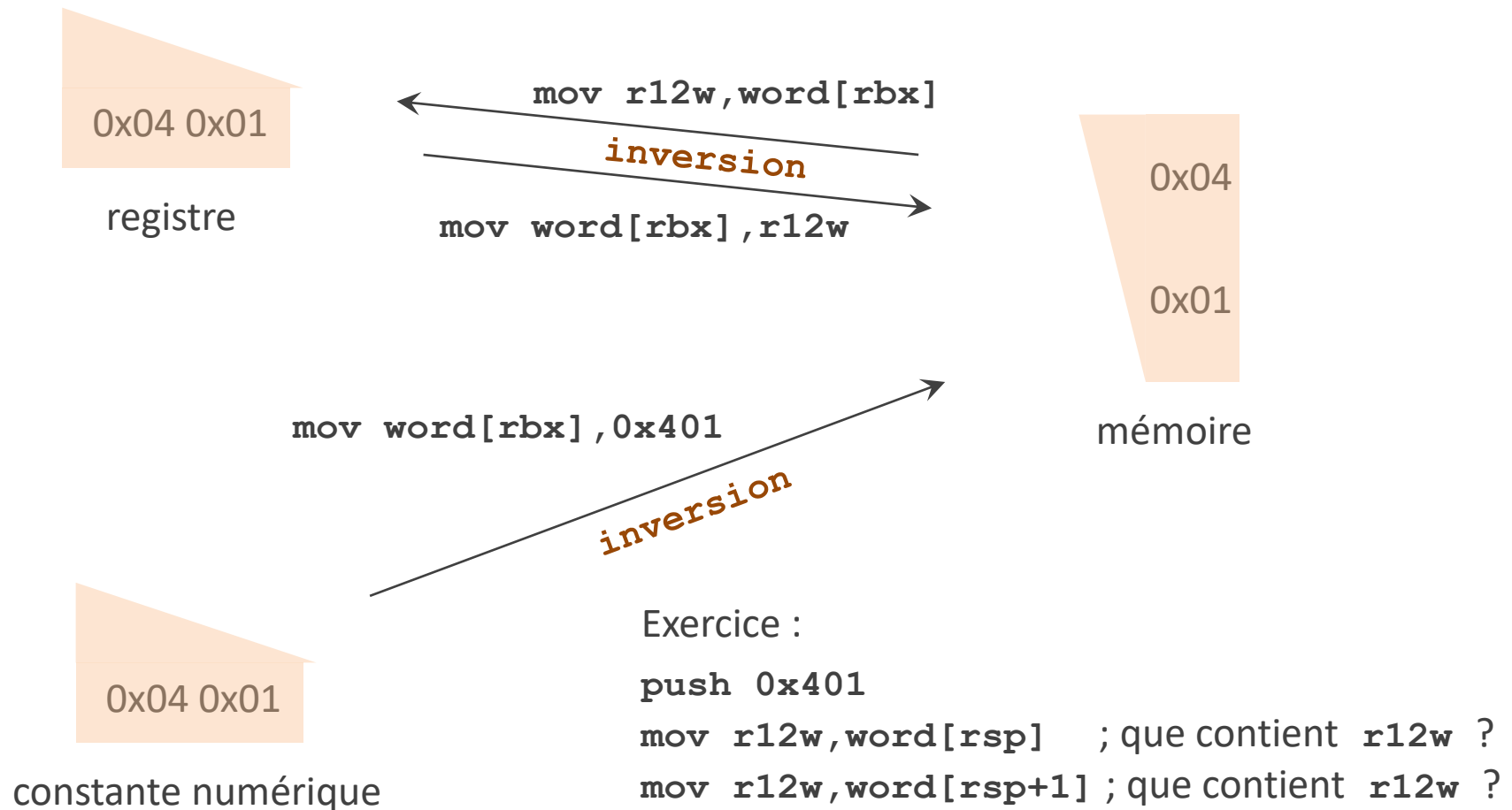
Notation mathématique des nombres

On note les chiffres de poids fort en premier (big-endian) et les chiffres de poids faible en dernier

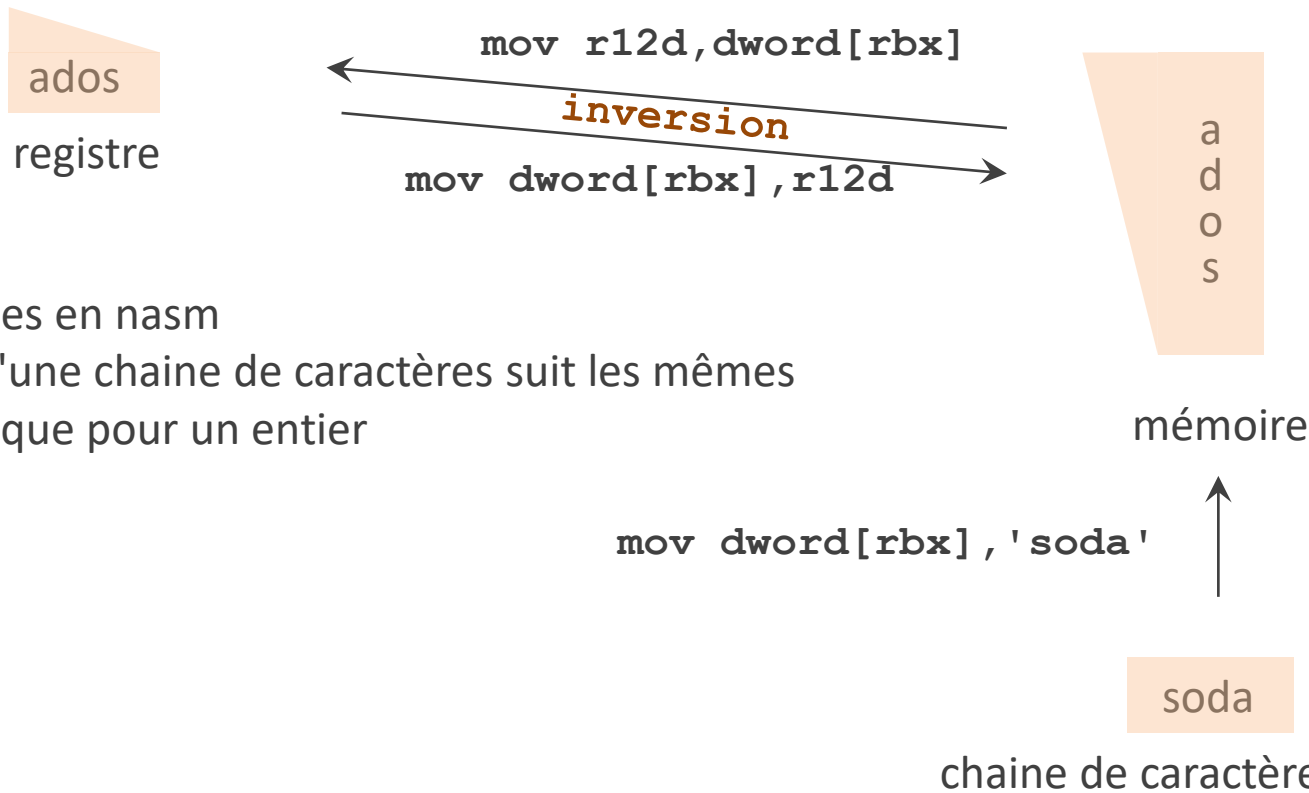
Registres

On utilise la même orientation

Orientation little-endian



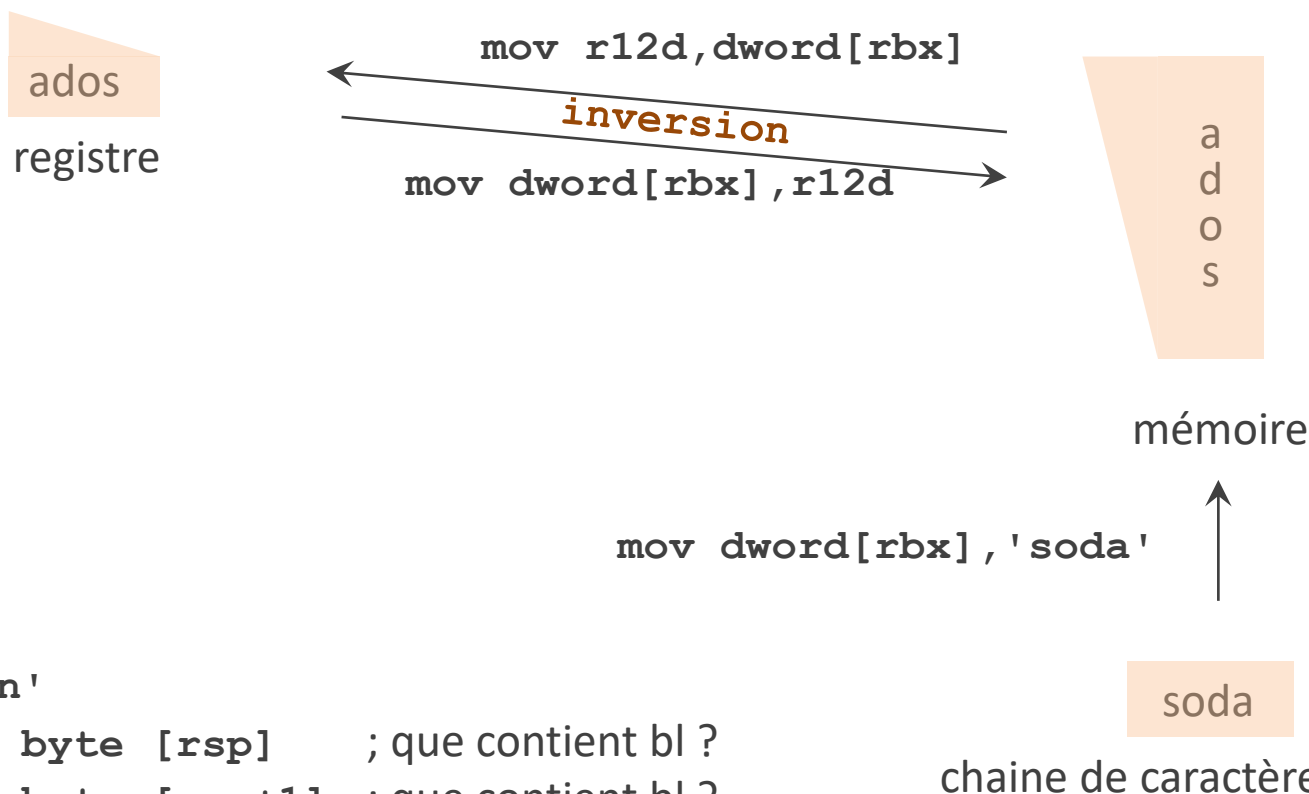
Orientation little-endian et chaines de caractères



Pas de types en nasm

La copie d'une chaîne de caractères suit les mêmes
règles que pour un entier

Orientation little-endian et chaines de caractères



Exercice :

```
push '\n'
mov bl, byte [rsp] ; que contient bl ?
mov bl, byte [rsp+1] ; que contient bl ?
```

Sommaire

Accès à la mémoire en écriture

La pile

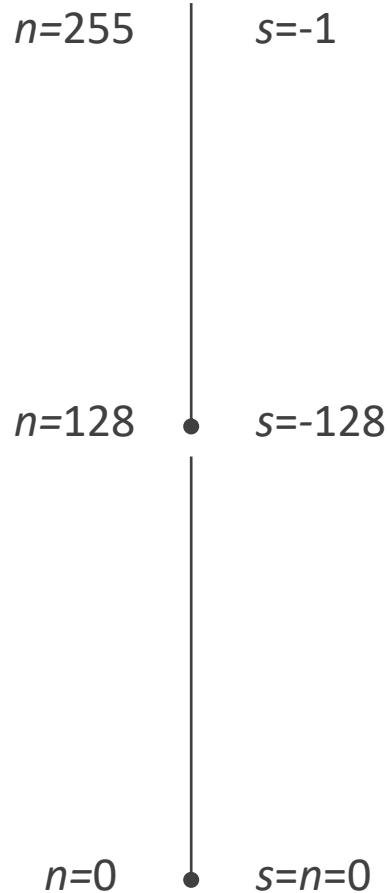
Orientation little-endian

Entiers signés et non signés

Instructions `imul` et `idiv`

Entiers signés et non signés sur 1 octet

Entiers non signés Entiers signés



Une même séquence de bits peut être interprétée soit comme entier non signé, soit en complément à 2

Entiers non signés

De 0 à 255

Entiers signés en complément à 2

Les entiers n de 128 à 255 sont interprétés comme $s=n-256$, donc entre -128 et -1

Entiers
non signés

Entiers signés

$$n=2^k-1$$

$$s=-1=n-2^k$$

$$n=2^{k-1}$$

$$s=-2^{k-1}=n-2^k$$

$$n=0$$

$$s=n=0$$

Bit de
poids
fort

1

0

Entiers non signés

Même principe sur plusieurs octets

On remplace 256 par 2^k avec $k>8$

$$s=n-2^k$$

Le bit de poids fort s'appelle aussi bit de
signe à cause des entiers signés

Représentation binaire de $-s$

Si $s<0$, on obtient la représentation binaire
de $-s$ à partir de celle de n

- en intervertissant les 1 et les 0
- puis en ajoutant 1

$$-s+n=2^k$$

Opérations communes aux entiers signés et non signés

```
mov rbx, qword 1
```

```
add bl, byte -1  
add bl, byte 255
```

Copie dans une destination de même taille

L'algorithme de `mov` est le même si le développeur veut utiliser l'entier n pour représenter n ou $s=n-2^k$

Addition, soustraction

L'algorithme de `add` est le même si le développeur veut utiliser l'entier n pour représenter n ou $s=n-2^k$

<code>jg</code>	jump if greater
<code>jl</code>	jump if less
<code>jge</code>	jump if greater or equal
<code>jle</code>	jump if less or equal
<code>ja</code>	jump if above
<code>jb</code>	jump if below
<code>jae</code>	jump if above or equal
<code>jbe</code>	jump if below or equal

```

mov rbx, 250    ; = 0x100 - 0x6
mov r12, 10
mov r13, 1
cmp bl, r12b    ; 0xfa, 0x0a
jg greater
    mov r13, 0
greater:        ; r13 contient 0
    
```

Opérations différentes pour les entiers signés et non signés

Multiplication et division

Pour les entiers signés : `imul`, `idiv`

Pour les entiers non signés : `mul`, `div`

Branchements conditionnels

Pour les entiers signés : `jg`, `jl`, `jge`, `jle`

Pour les entiers non signés : `ja`, `jb`, `jae`, `jbe`

On peut initialiser un entier comme positif et l'utiliser comme négatif

Constantes entières négatives

```
section .data
my_negb:  db -12
my_neg2b: dw -12
my_neg4b: dd -12
my_neg8b: dq -12
section .text
mov rbx, 0
mov bl, [my_negb]
mov r12, 0
mov r12w, [my_neg2b]
mov r13, 0
mov r13d, [my_neg4b]
mov r14, [my_neg8b]
call show_registers
```



```
section .data
my_diff: dq -2
```

met à 1 les 63 bits de poids
fort de la zone allouée

Le nombre de bits de poids fort à 1 dépend de la
taille de la zone

Comparaison avec la base 10

$$100 - 6 = 94$$

$$10\ 000 - 6 = 9\ 994$$

$$1\ 000\ 000 - 6 = 999\ 994$$

rbx:0xf4 r12:0xffff4 r13:0xfffffffff4 r14:0xfffffffffffffffff4

Extension d'entiers

Mettre un entier sur une zone mémoire plus étendue sans changer sa valeur

Extension par zéro

Ne change pas la valeur comme entier non signé

```
mov ebx, r12d
```

met à **zéro** les 32 bits de poids fort de **rbx**

0xfd0a_74c3 devient 0x0000_0000_fd0a_74c3

Extension de signe

Ne change pas la valeur comme entier signé

Copie du bit de plus haut rang ou bit de signe

Exemple : **push** avec une constante qui tient sur moins de 8 octets

```
movsx rbx, r12d
```

0xfd0a_74c3 devient 0xffff_ffff_fd0a_74c3

Constante sur plus de 4 octets

```
add rbx,          0xfd0a_74c3 ; OK
add rbx,          0x3_fd0a_74c3 ; tronqué puis extension de signe
add rbx, qword 0x3_fd0a_74c3 ; tronqué puis extension de signe
mov rbx,          0xfd0a_74c3 ; OK
```

La plupart des instructions refusent que le 2^e opérande soit une constante sur plus de 4 octets, même si le 1^{er} opérande est sur 8 octets

Si le 1^{er} opérande est sur 8 octets

les constantes sont **tronquées** à 4 octets (on perd les 8 octets de poids fort) puis étendues par extension **de signe**

```
mov rbx,          0x3_fd0a_74c3 ; OK
mov rbx, qword 0x3_fd0a_74c3 ; OK
```

Exception

```
mov r64, imm64
```

Sommaire

Accès à la mémoire en écriture

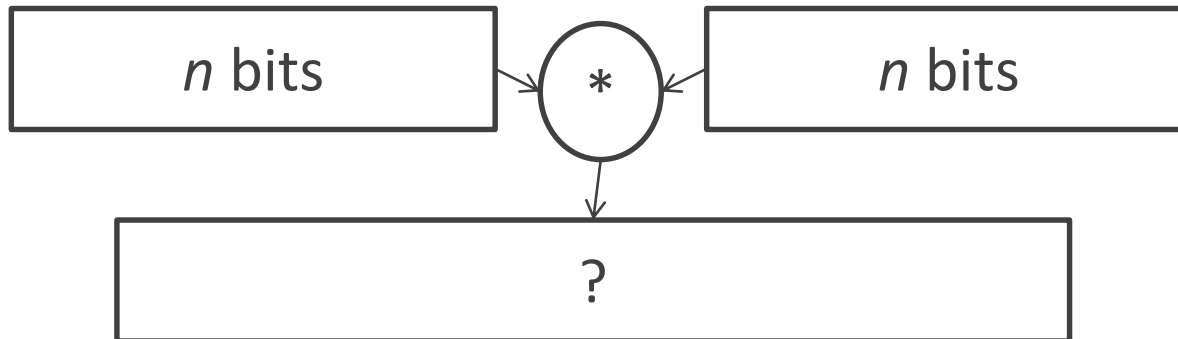
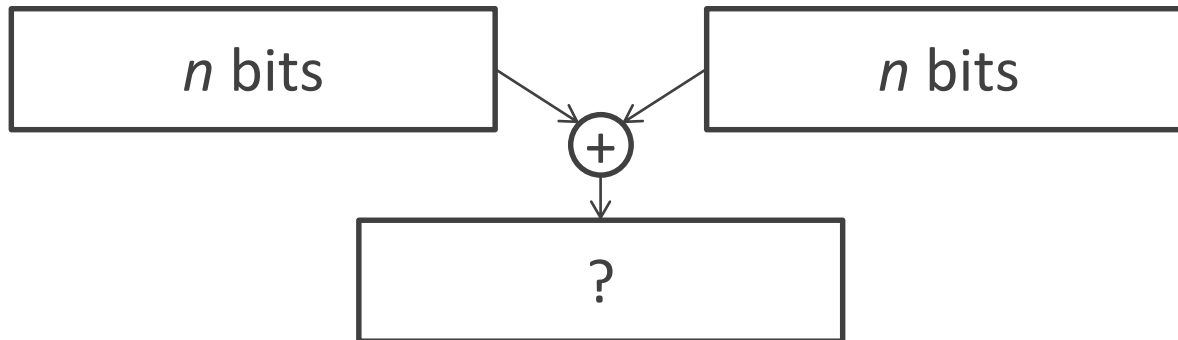
La pile

Orientation little-endian

Entiers signés et non signés

Instructions **`imul`** et **`idiv`**

Addition et multiplication



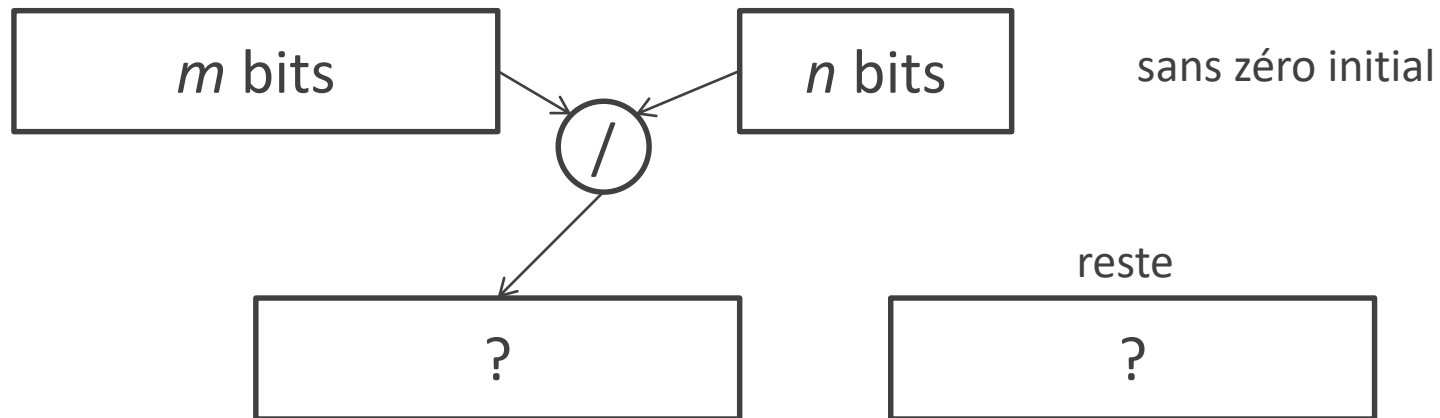
Multiplication

```
mov    rbx, 90
mov    r12, 9
imul   rbx, r12    ; rbx := rbx * r12
call   show_registers
```

Les opérandes sont interprétés comme entiers
signés

Si la valeur absolue du produit ne tient pas sur 8
octets, il est **tronqué** : on perd les 8 octets de
poids fort

Division entière



Division entière

```

mov     rdx, 0
mov     rax, 90
mov     rbx, 9
idiv    rbx ; divise rdx:rax par rbx
mov     r13, rax
mov     r12, rdx
call    show_registers
    
```

L'instruction `idiv` utilise `rax` et `rdx` comme registres spéciaux

L'instruction `idiv` prend un seul opérande : le diviseur

Si le diviseur est sur 8 octets

L'entier qui va être divisé (`rdx:rax`) est obtenu en concaténant les 8 octets de `rdx` avec ceux de `rax`

Autre exemple : `rdx=0x87`, `rax=0x41a`,
`rdx:rax=0x87_0000_0000_0000_041a`

Le quotient est placé dans `rax`

Le reste est placé dans `rdx`

Les résultats écrasent `rax` et `rdx`

Division entière

```

mov     rdx, 0
mov     rax, 90
mov     rbx, 9
idiv    rbx      ; divise rdx:rax par rbx
mov     r13, rax
mov     r12, rdx
call    show_registers

```

Les entiers sont interprétés comme entiers signés
 Le diviseur peut être un accès mémoire, mais pas
 une constante

Pour diviser par une constante, la copier dans un
 registre

Si le diviseur est sur 2 ou 4 octets

L'entier qui va être divisé est obtenu en
 concaténant certains octets de `rdx` avec
 certains octets de `rax`

Exercice

Vérifier si les 6 octets de poids fort de `rax` sont à zéro

Division entière

```
mov     rdx, 0
mov     rax, 90
mov     rbx, 9
idiv    bx      ; divise ax par bx
```

Exemple	taille de x	divise...	par...	met le quotient dans...	et le reste dans...
<code>idiv bl</code>	1 octet	<code>ax</code>	<code>x</code>	<code>al</code>	<code>ah</code>
<code>idiv bx</code>	2 octets	<code>dx:ax</code>	<code>x</code>	<code>ax</code>	<code>dx</code>
<code>idiv ebx</code>	4 octets	<code>edx:eax</code>	<code>x</code>	<code>eax</code>	<code>edx</code>
<code>idiv rbx</code>	8 octets	<code>rdx:rax</code>	<code>x</code>	<code>rax</code>	<code>rdx</code>

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR